



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING

Descriptive Written Exam

Name :- M Sivam Kumar

Register Number :- 192521017

COURSE NAME :Software Testing

COURSE CODE: CSA3712

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3

3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3

8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3

15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I **Sivam Kumar(192521017)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

SIVAM KUMAR.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWERS

1. Engineering Approach to Software Testing

Software testing should be carried out as a planned and systematic engineering process instead of random testing.

- First, the team should understand the banking requirements and identify important functions.
- A proper test plan should be prepared with scope, schedule, resources, risks, and responsibilities.
- Test cases should be designed using techniques such as equivalence partitioning, boundary value analysis, decision tables, and state-transition testing.
- Expected results should be defined before executing each test case.
- Test-management and defect-tracking tools can be used to maintain testing information.
- Common standards should be followed for test cases, defect severity, priority, and reporting.
- Risk-based testing should give more attention to critical banking operations.
- For example, fund transfer should be tested with valid accounts, insufficient balance, invalid account numbers, transaction limits, and network problems.
- After fixing defects, retesting and regression testing should be performed.
- This systematic approach increases test coverage, detects defects early, and improves software quality.

2. Structured Testing Process Across SDLC

A proper testing process should be followed during different stages of the SDLC.

- During requirement analysis, the tester identifies all requirements related to ATM withdrawal.
- During test planning, the testing scope, resources, risks, and schedule are decided.
- During test design, positive, negative, and boundary test cases are prepared.

- During test execution, actual results are compared with expected results.
- If a problem is found, it should be recorded in the defect repository.
- Defects should be given suitable severity and priority.
- After correction, the tester performs retesting.
- Regression testing is performed to ensure that existing ATM functions are not affected.
- Test cases include valid withdrawal, wrong PIN, insufficient balance, invalid amount, daily limit, and cash unavailability.
- Following the same process throughout the SDLC reduces mistakes and improves ATM reliability.

3. Error, Defect and Failure Relationships

Error, defect, and failure are three different stages of a software problem.

- An error is a mistake made by a person during requirements, design, coding, or testing.
- A defect is an incorrect part of the software caused by that error.
- A failure occurs when the defect is executed and the system gives an incorrect result.
- For example, a developer may write the wrong password validation logic.
- The developer's mistake is called an error.
- The incorrect password logic in the program is the defect.
- When a genuine user cannot log in because of this defect, it is a failure.
- Defect reports should clearly describe the problem, steps to reproduce it, expected result, and actual result.
- Correctly understanding these terms helps testers and developers communicate clearly.
- It also makes defect tracking and resolution more effective.

4. Defect Clustering and Test Case Prioritization

Defect clustering means that many defects are usually found in a small number of modules.

- Previous defect records can be studied to identify modules where defects occur frequently.
- These modules should receive more testing attention.
- Critical business functions should also be given high priority.
- Test cases can be prioritized according to risk, complexity, business importance, and previous defect history.
- Recently changed modules should receive additional regression testing.
- For example, in an online shopping application, payment and checkout are more critical than the user-profile section.
- High-risk test cases should be executed first.
- Low-risk areas can be tested after the important functions.
- This allows testers to use their time and resources more effectively.
- Defect clustering and prioritization improve defect detection and overall testing efficiency.

5. Defect Classification and Repository Management

Defects can occur at different stages of software development.

- They may originate from requirements, design, coding, integration, or configuration.
- Defects can be grouped into functional, performance, security, usability, interface, compatibility, and data-related categories.
- A defect repository is used to store information about discovered defects.
- It may contain defect ID, description, module, severity, priority, status, root cause, and solution.
- Previous defect information helps testers find repeated problem areas.

- If the same problem appears in different releases, the repository can help identify the pattern.
- New regression test cases can be prepared from important old defects.
- Root-cause analysis should be performed for recurring defects.
- Code reviews and preventive testing can help stop similar defects from occurring again.
- Proper use of the repository improves future testing and overall software quality.

6. Systematic Testing of Mobile Banking Fund Transfer

Frequent crashes during fund transfer indicate that the application needs more systematic testing.

- First, the tester should reproduce the crash and collect logs and other useful information.
- The tester should check the device, operating system, application version, network, and transaction details.
- Normal fund-transfer operations should be tested first.
- Negative cases such as insufficient balance and invalid account numbers should also be tested.
- Large transaction amounts and transaction limits should be checked.
- Network disconnection during a transfer should be tested.
- Session timeout and repeated transactions should also be considered.
- Integration testing should verify communication between the application, database, and banking services.
- After the defect is fixed, retesting and regression testing should be performed.
- Systematic testing helps identify the cause of crashes and reduces similar failures in future releases.

7. Testing and Debugging: Roles and Differences

Testing and debugging are different activities even though they work together.

- Testing is used to find defects and check whether the software satisfies its requirements.
- Debugging is the process of finding the cause of a defect and correcting the program.
- Testing mainly identifies that something is wrong.
- Debugging determines where the problem exists and how it can be corrected.
- For example, testing may show that the balance becomes incorrect after a fund transfer.
- The developer then debugs the program to find the incorrect calculation.
- Once the defect is corrected, the tester performs retesting.
- Regression testing is then performed to check whether other functions still work correctly.
- Confusing testing with debugging can result in poor defect handling.
- The proper sequence is Testing → Defect Report → Debugging → Retesting → Regression Testing.

8. Cross-Browser Compatibility Testing

Cross-browser testing checks whether a web application works properly on different browsers.

- The application should be tested on browsers such as Chrome, Firefox, Edge, and Safari where required.
- Different browser versions and operating systems may also need to be considered.
- Important functions such as login, forms, navigation, and transactions should be tested.
- The appearance of buttons, menus, text, and other interface elements should also be checked.
- Browser-specific JavaScript or CSS behavior may create defects.
- Responsive behavior should be checked on different screen sizes.

- A browser compatibility matrix can help organize the testing process.
- Automation tools such as Selenium can reduce the effort required for repeated browser testing.
- Defects should mention the browser and environment in which they occurred.
- Systematic cross-browser testing improves compatibility, reliability, and user experience.

9. Classification of Payroll Logical Defects

Logical errors occur when a program runs successfully but produces an incorrect answer.

- Payroll defects may occur in salary, overtime, tax, deduction, and bonus calculations.
- These defects can be classified as calculation, business-rule, data, or requirement defects.
- Understanding the defect type helps testers select the right testing technique.
- Normal salary calculations should be tested.
- Minimum and maximum salary values should also be checked.
- Overtime, tax, deductions, bonuses, and different employee conditions should be tested.
- Unit testing can detect calculation problems at an early stage.
- Code reviews can help identify incorrect business logic.
- Boundary value analysis and equivalence partitioning can provide better coverage.
- Regression testing should be performed after fixing the calculation defect.
- Proper classification and testing help prevent similar payroll errors in future releases.

10. Structured Test Case Design Techniques

Random testing does not provide complete or reliable test coverage.

- Testing should begin with requirements and risk identification.

- Every test case should clearly contain the test ID, objective, preconditions, inputs, steps, expected result, and status.
- Equivalence partitioning divides inputs into different valid and invalid groups.
- Boundary value analysis checks values near the limits.
- Decision-table testing is useful when several conditions affect the result.
- State-transition testing is useful when the system changes from one state to another.
- Risk-based testing gives greater importance to critical functions.
- For example, a bank transaction limit should be tested below, exactly at, and above the permitted limit.
- Structured test cases make testing easier to repeat, review, and maintain.
- They improve coverage and increase the chances of finding defects.
- Therefore, planned test-case design is much more effective than random testing.

11. Healthcare Data Integrity Testing

Data inconsistency occurs when different modules do not correctly store, update, or display the same information.

- It may be caused by incorrect requirements, programming mistakes, database problems, or integration defects.
- Testing should check whether data remains correct across all modules.
- Patient details can be compared between registration, medical records, billing, and reporting.
- Database testing should verify correct data storage and retrieval.
- Integration testing should check communication between modules.
- Both valid and invalid data should be tested.
- Updates and transactions should also be checked carefully.
- Every data-related defect should be properly documented.

- Regression testing should be performed after fixing the problem.
- Systematic testing helps maintain data consistency and protects the accuracy of healthcare information.

12. Defect Repository Analysis and Recurring Defect Prevention

A defect repository helps the testing team maintain a history of problems found in different releases.

- It should contain information such as defect ID, module, type, severity, priority, root cause, and resolution.
- Repository information can be analyzed to find frequently failing modules.
- Defects can also be studied according to release and category.
- Repeated defects may indicate that the original root cause was not completely solved.
- Important previous defects should be converted into regression test cases.
- Frequently failing modules should receive more testing attention.
- Root-cause analysis should be performed for recurring problems.
- Code reviews can help prevent the same mistake during future development.
- Automated regression testing can check whether old defects have returned.
- Repository analysis provides useful information for future test planning.
- Proper use of historical defect data reduces repeated problems and improves software quality.

13. Performance Testing for Peak-Usage Scenarios

Failure during peak usage shows that the system was not sufficiently tested under heavy workload.

- Test planning should identify the expected number of normal and maximum users.
- Load testing checks the system under expected user load.

- Stress testing checks the system beyond its normal capacity.
- Performance testing measures response time, throughput, and resource usage.
- Multiple users should access the application simultaneously during testing.
- Important functions such as login, course access, examinations, and result submission should be tested.
- Database and server performance should also be monitored.
- For example, thousands of students may access an examination system at the same time.
- Such peak conditions should be simulated before deployment.
- Performance defects should be recorded and analyzed.
- Better planning and test coverage improve system reliability and reduce failures after deployment.

14. Early Tester Involvement in SDLC

Testers should not wait until the software is completely developed before becoming involved.

- During requirement analysis, testers can find incomplete, unclear, or conflicting requirements.
- During the design stage, they can identify risks and testability problems.
- During development, they can support unit and integration testing.
- Test cases can be prepared before the complete application is ready.
- Finding defects early reduces the time and cost required for corrections.
- Regular communication between testers and developers improves understanding.
- For example, in a banking system, security and transaction requirements can be reviewed before implementation.
- Early testing prevents incorrect requirements from moving into later stages.
- It also improves test planning and coverage.

- Early tester involvement reduces rework and improves the quality of the final product.

15. Improving Test Coverage

Incomplete test coverage means that some important conditions or requirements have not been tested.

- Test coverage should be planned based on requirements and application risks.
- Functional as well as non-functional requirements should be considered.
- Positive and negative test cases should both be included.
- Equivalence partitioning can be used to test different input groups.
- Boundary value analysis can be used to check values near limits.
- Edge cases and invalid inputs should also be tested.
- For example, ATM withdrawal testing should include valid amount, zero, negative amount, maximum limit, and insufficient balance.
- Regression testing should ensure that existing functionality continues to work.
- Test coverage should be reviewed before completing the testing phase.
- Missing test cases may allow defects to reach production.
- Good coverage increases confidence in the reliability and correctness of the software.

16. Effective Defect Reporting

Good defect reporting helps the development team understand and fix problems quickly.

- Each defect should have a unique identification number.
- The report should contain a clear title and the affected module.
- Application version and testing environment should be mentioned.
- The exact steps required to reproduce the defect should be provided.
- The expected result and actual result must be clearly separated.

- Severity and priority should be assigned correctly.
- Screenshots, error messages, and logs can be attached as supporting evidence.
- For example, “Transaction fails when the amount exceeds the allowed limit” is clearer than “Transaction problem.”
- A centralized defect-tracking system should be used.
- Standard reporting formats should be followed by all testers.
- Clear defect reports reduce confusion and help developers resolve issues faster.

17. Process-Based Testing for Consistency

Process-based testing ensures that all modules follow a common and organized testing procedure.

- A standard process should be defined for the complete project.
- The process can include Requirement Analysis → Test Planning → Test Design → Test Execution → Defect Reporting → Retesting → Regression Testing → Test Closure.
- All testers should use a common test-case format.
- Standard definitions should be followed for defect severity and priority.
- Test plans and test cases should be reviewed before execution.
- Checklists can help testers avoid missing important activities.
- For example, all modules of a banking application should follow the same method for recording and tracking defects.
- Standard processes make testing more consistent and repeatable.
- They also improve communication between testers and developers.
- Process-based testing improves quality, traceability, and control over the testing activities.

18. Boundary Value Analysis and Edge Case Testing

Edge cases are conditions that occur at or near the limits of the allowed input.

- Testers may miss these cases when they focus only on normal inputs.
- Boundary Value Analysis is useful for finding defects at input limits.
- Values just below, at, and just above a boundary should be tested.
- For example, if the accepted age is from 18 to 60, test 17, 18, 19, 59, 60, and 61.
- The same technique can be applied to transaction limits, file sizes, quantities, and dates.
- Minimum and maximum allowed values should be tested.
- Zero, empty, invalid, and unusually large values can also be tested when appropriate.
- Edge cases should be included during test-case design.
- Boundary testing can identify problems that normal test cases may not find.
- It improves test coverage and reduces the possibility of failures after deployment.
- Therefore, boundary testing is an important part of effective test design.

19. Defect Documentation and Repository Management

Proper defect documentation makes it easier to monitor and manage software problems.

- Each defect should have a unique ID and a clear description.
- The repository should contain the module, environment, reproduction steps, expected result, actual result, severity, priority, and status.
- The root cause and final solution should also be recorded.
- Defects can be tracked from New → Assigned → Fixed → Retested → Closed.
- Historical records help identify modules where defects occur repeatedly.
- Common defect patterns can be studied to find their causes.
- Previous defects can be converted into regression test cases.
- Repository information can help testers plan future testing.

- Proper documentation prevents important defect information from being lost between releases.
- It also improves communication between developers, testers, and managers.
- A well-maintained repository supports better analysis, prevention, and quality improvement.

20. Comprehensive Banking Transaction Testing

Banking transaction processing requires careful testing because failures can affect both customers and financial information.

- Testing should begin with requirement analysis and identification of transaction-related risks.
- Positive and negative test cases should be prepared.
- Test successful transactions, insufficient balance, invalid account details, and transaction limits.
- Duplicate transactions, network interruptions, and timeout conditions should also be tested.
- Techniques such as equivalence partitioning, boundary value analysis, decision tables, and state-transition testing can be used.
- Integration testing should verify communication between the application, database, and transaction services.
- Security-related conditions such as authentication and authorization should also be tested.
- Every defect should be recorded with proper severity, priority, and reproduction steps.
- Retesting should be performed after fixing defects.
- Regression testing should ensure that existing transaction functions still work correctly.
- Automated regression testing can improve repeated test execution.